

# Zonal Architecture 기반 차량 제어 시스템 프로젝트

TOPST AI-G, D3-G, VCP 보드 활용  
Zonal Architecture 시스템 고도화

26.05.28 / 정예찬

# 1. 프로젝트 개요



## 주제

- Zonal Architecture  
기반 RC 카 제어 시스템 및  
3차선 AI 레이더(FSD)  
대시보드 구현



## 목표

- 차량 내 네트워크(Ethernet,  
CAN)를 통해  
ADAS(인지), HPC(판단),  
MCU(제어)를 분리하고  
통합하는 시스템 구축 및 고도화



## 개발 환경

개발 보드:  
TOPST D3-G(Linux 환경)  
TOPST AI-G  
TOPST VCP

# 1. 프로젝트 개요

## 주제 선정 이유:

### 기존 DashBoard 한계

#### 제한적 구동

특정 Linux(Ubuntu) 환경에서만  
제한적으로 구동되어 범용성이  
떨어졌음.

#### 차량 기능 제어 불가

- 단순히 정보를 띄워주는 역할만 할 뿐,  
실제 차량의 기능 제어 불가
- 기능 제어를 위해선 별도의 터미널 ->  
Command 코드 입력

#### 차선 인식 기능 block

기본 제공된 시스템에서는 차선 인식  
기능이 막혀 있어 차선 기반의 공간  
인지 불가.



### 개선 DashBoard

#### Web-based

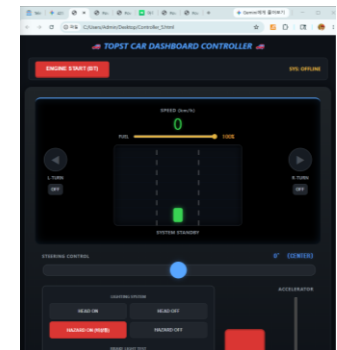
Web Bluetooth / Web Serial  
API를 활용하여 웹 브라우저에서  
구동

#### 하드웨어 실시간 직접 제어

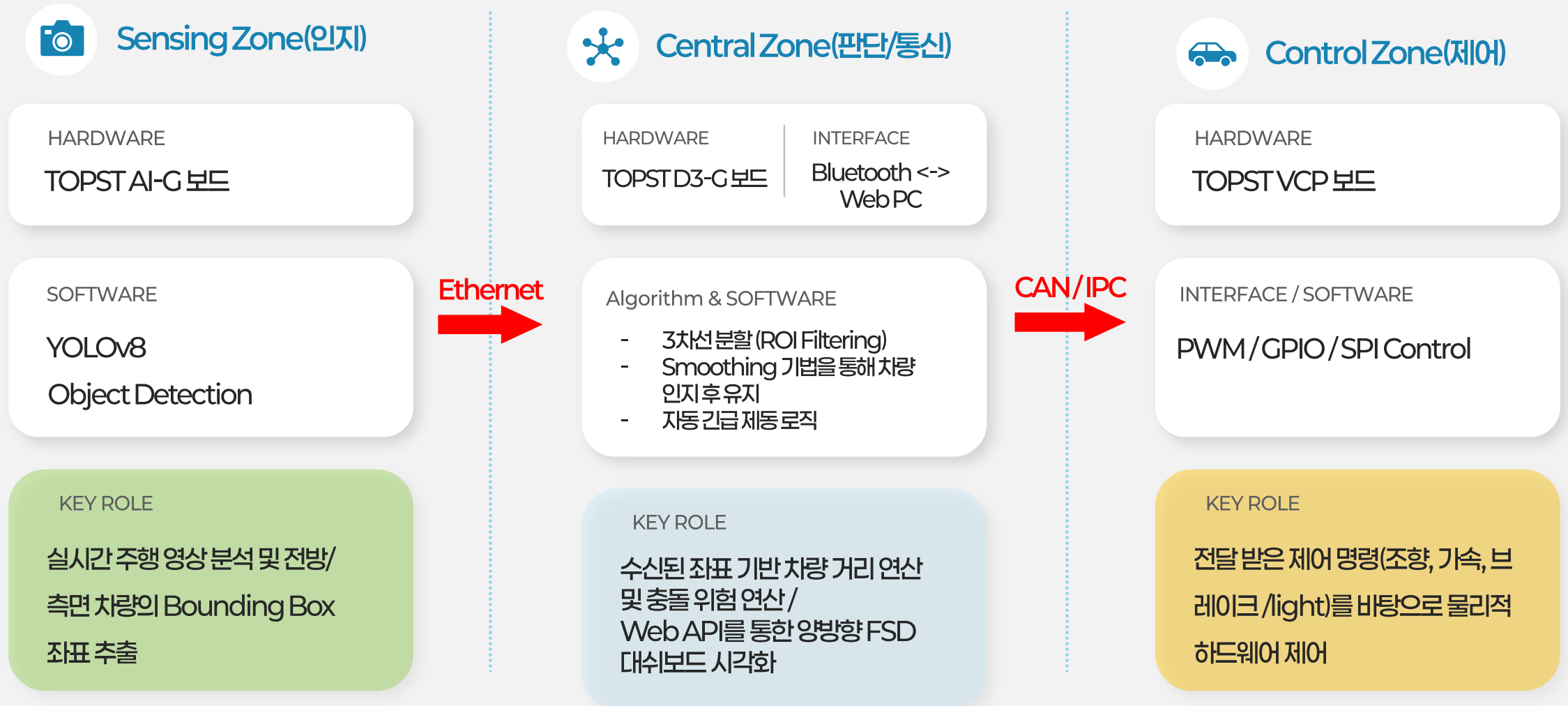
웹 UI 상에서  
하드웨어(모터, 브레이크, LED)를  
실시간으로 직접 제어 및 피드백

#### 차선 인식 기능

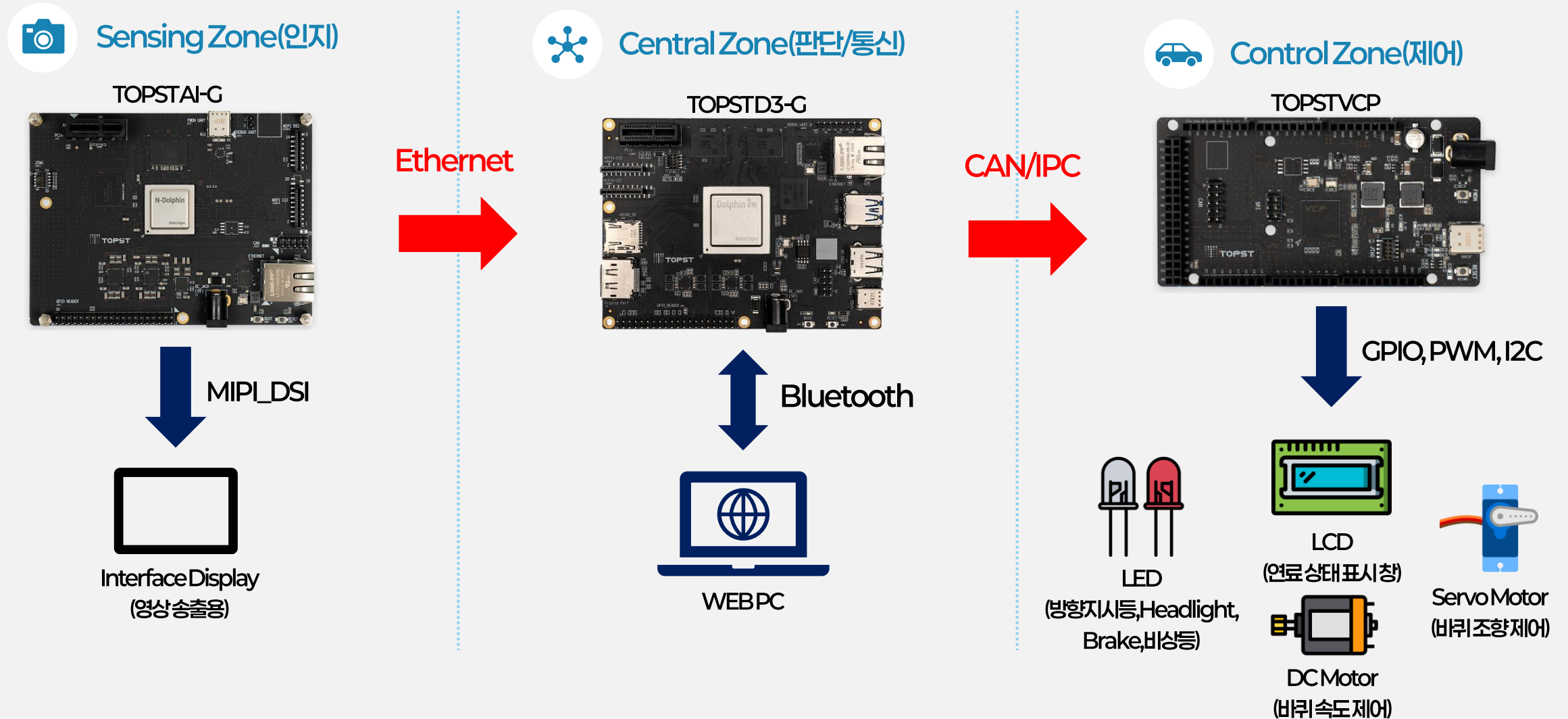
YOLO 데이터의 X/Y 좌표를  
재해석하여 3차선 영역을  
논리적으로 분할 및 시각화



## 2. 시스템 아키텍처



## 2.1 하드웨어 구성



## 3. 트러블 슈팅

### (1) 차선인식한계개선-3차선구현/객체추적

```
for obj in objects:
    # 1. Y좌표로 박스 높이 계산
    y1 = float(obj.get("y_min", obj.get("ymin", obj.get("y1", 0))))
    y2 = float(obj.get("y_max", obj.get("ymax", obj.get("y2", 0))))
    box_height = abs(y2 - y1)

    # 2. X좌표로 차량 가로 위치(중앙값) 계산
    x1 = float(obj.get("x_min", obj.get("xmin", obj.get("x1", 0))))
    x2 = float(obj.get("x_max", obj.get("xmax", obj.get("x2", 0))))
    x_center = (x1 + x2) / 2.0

    # 3. 박스 높이를 거리(m)로 환산
    if box_height > 0:
        dist = max(2.0, 50.0 - (box_height * 0.25))
    else:
        dist = 15.0

    # 4. 차선(ROI) 분류 및 최단 거리 갱신 (해상도 1280 기준)
    if x_center < 500:
        if dist < cur_dist_L: cur_dist_L = dist      # [좌측 차선]
    elif x_center > 780:
        if dist < cur_dist_R: cur_dist_R = dist      # [우측 차선]
    else:
        if dist < cur_dist_C: cur_dist_C = dist      # [내 차선(중앙)]
```

#### 기존 시스템 한계

AI-G yolo를 통해 추출한 단일 Bounding Box로는 내 차선과 옆 차선의 차량을 구분할 수 없음, 옆 차선 차량에도 긴급 제동이 개입하는 오류 발생

#### 코드 구현 방안

비활성화된 딥러닝 차선 인식 모델을 대체하기 위해, 카메라 화각을 수학적으로 분할하는 Static ROI 알고리즘 개발 및 좌/중/우 좌표 필터링 적용

#### 결과

- 차선 신뢰성 확보: 오직 정면 차량 조우 시에만 AEB 반응
- Multi-Tracking: 3차선의 가장 가까운 차량들을 각각 동시 추적

## 3. 트러블 슈팅

### (2) 차량인식개선-Smoothing

```
# 💡 [스무딩 로직 적용: 기억하고 버티기]

# 1. 좌측 차선
if cur_dist_L < 100:
    last_L = cur_dist_L
    ttl_L = MAX_TTL      # 데이터 갱신
                        # 수명 리셋
else:
    if ttl_L > 0: ttl_L -= 1 # 안 보이면 수명 1 깎음
    else: last_L = 999.0   # 수명이 다 되면 진짜로 삭제

# 2. 중앙 차선
if cur_dist_C < 100:
    last_C = cur_dist_C
    ttl_C = MAX_TTL
else:
    if ttl_C > 0: ttl_C -= 1
    else: last_C = 999.0

# 3. 우측 차선
if cur_dist_R < 100:
    last_R = cur_dist_R
    ttl_R = MAX_TTL
else:
    if ttl_R > 0: ttl_R -= 1
    else: last_R = 999.0
```

#### 기존 시스템 한계

AI-G YOLO가 객체를 정확하게 인식하지 못하고, 객체를 한번 잡고 금방 놓침(박스가 깜빡이는 것처럼 보임).  
-> 데이터가 즉각 소멸되어 UI 상에 깜빡거려 잘 나타내지 못함

#### 코드 구현 방안

AI YOLO 알고리즘의 신뢰도를 낮추는 접근 대신, 시스템 상으로 인식한 객체를 기억하도록 하고, 찰나의 미인식 구간에서도 직전 데이터를 유지하도록 구현

#### 결과

- 대쉬보드 UI 안정성: 인식한 타겟 점의 깜빡임 해결
- 타겟 인식 개선: 불필요한 제동 간섭 없어짐

## 3. 트러블 슈팅

### (3) Web UI 시각화 고도화 - 원근감 스케일링

```
function updateLaneDot(elementId, distance) {  
  let dot = document.getElementById(elementId);  
  if (distance > 0 && distance < 100) {  
    dot.style.display = 'block';  
  
    // 💡 [원근감 튜닝 변수]  
    // 이 숫자를 1.5, 1.8, 2.0 등으로 키울수록 레이더 상에서 차가 훨씬 멀리(위로) 밀려남.  
    // 기본값인 1.0일 때보다 1.6배 더 멀리 보이도록 세팅  
    let visualScale = 1.6;  
  
    // 거리 * 가중치 + 기본 여백(5)을 더해 위치를 계산.  
    let pos = Math.max(15, Math.min((distance * visualScale) + 5, 90));  
  
    dot.style.bottom = pos + '%'  
  } else {  
    dot.style.display = 'none';  
  }  
}
```

#### 기존 시스템 한계

초기 HTML 레이더 도입 시, 거리를 단순 선형으로 계산하여 원거리의 차량이 대시보드 상에서 지나치게 가깝게 표시됨

#### 코드 구현 방안

거리 데이터를 Bounding Box의 y좌표의 크기를 통해 계산하고 산출한 거리 데이터에 시각적 스케일 상수를 곱하여 공간감을 사실적으로 재현하려고 하였음

#### 결과

- 대쉬보드 UI 안정성 : 실제 주행 영상과 유사한 랜더링
- UI 동기화 완성

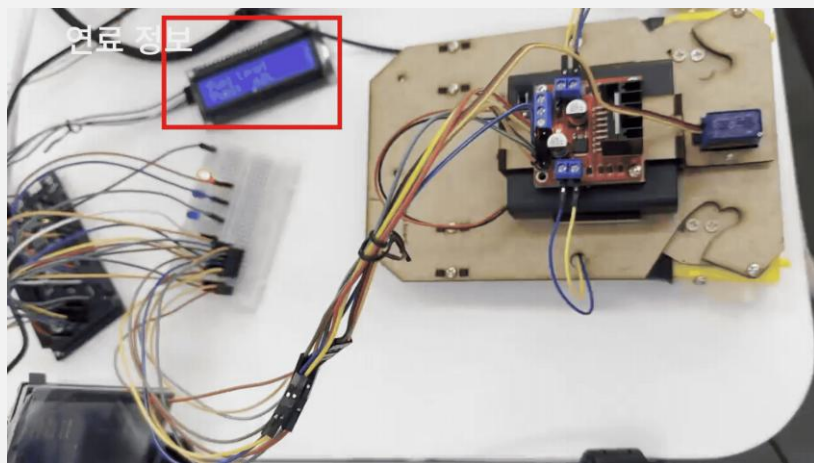


## 4. 동작 영상

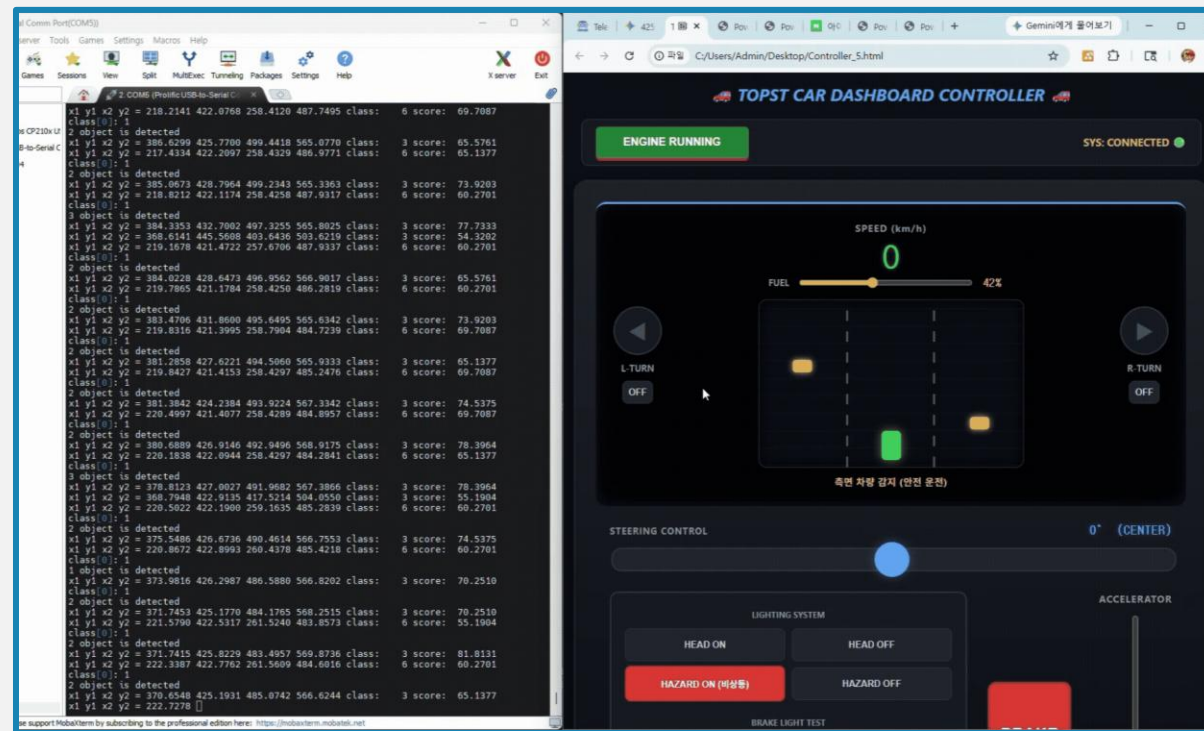
디스플레이 화면 – 객체 인식



하드웨어 동작



대쉬보드 동작 영상



## 5. 프로젝트 고찰

### Zonal Architecture의 실제적 이해와 SDV 가치 증명

- 인지 -> 판단 -> 제어 영역을 이더넷과 CAN 통신으로 유기적 결합
- 소프트웨어가 하드웨어 동작을 유연하게 정의하고 제어하는 SDV 구현

“단순한 모듈의 연결을 넘어, 하드웨어 계층의 통신 최적화부터 웹 UI 렌더링까지 직접 트러블슈팅하며 자율주행 통합 제어 시스템의 완성도를 입증한 엔지니어링 경험”

### 소프트웨어 신뢰성 고도화

- 공간 인지 한계 극복
- 딥러닝 모델의 프레임 누락을 보정하여 객체 인식률을 소프트웨어적으로 높임

### 하드웨어 동기화 및 물리적 검증

- 프로토콜 및 버퍼 최적화
- 실제 구동 모터, 브레이크 점등이 딜레이 없이 즉각 반응하는지 철저한 교차 검증